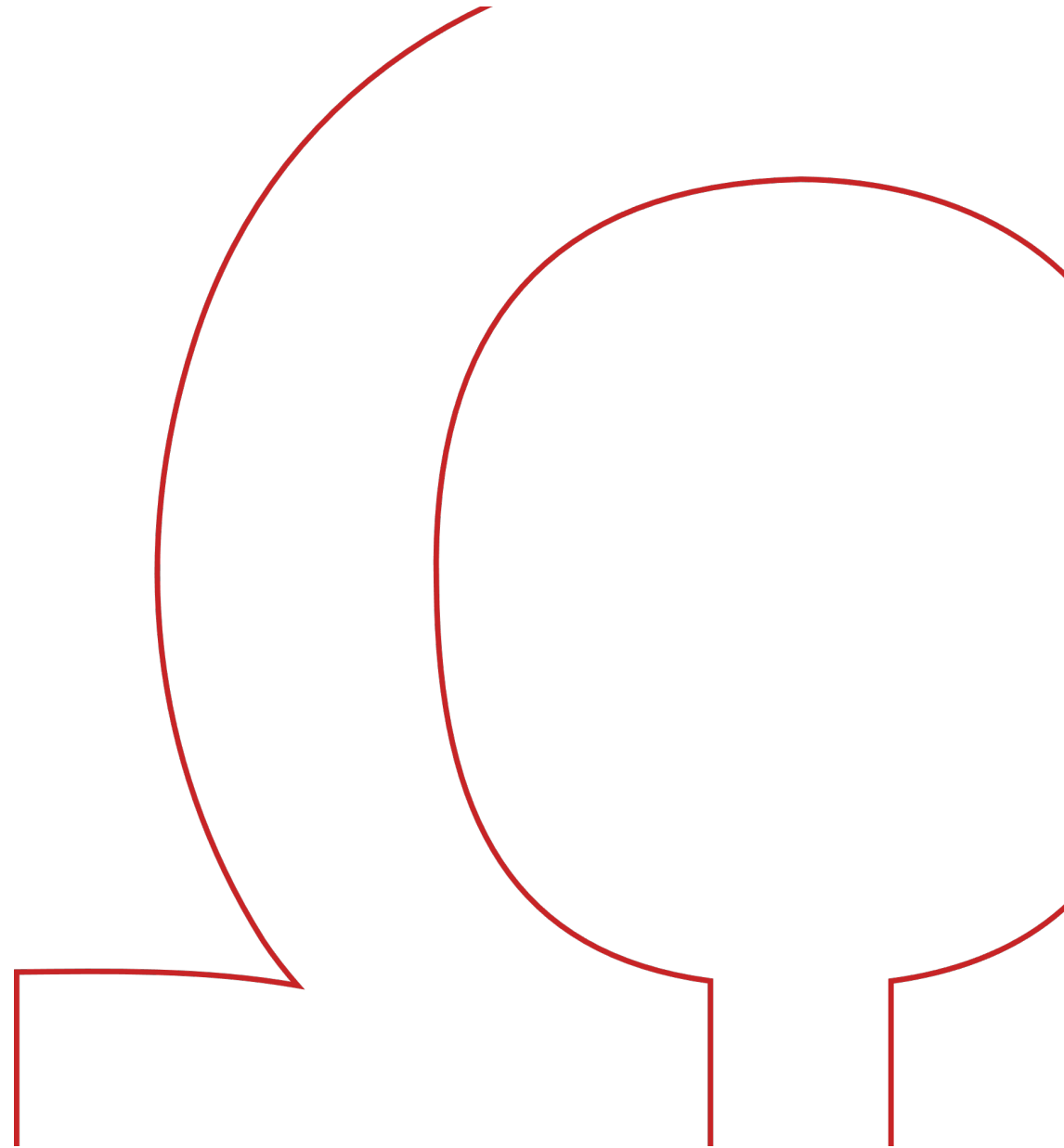


Prog III / A

Software Engineering



Unified Modeling Language (UML)

Unified Modeling Language (UML)

Definition

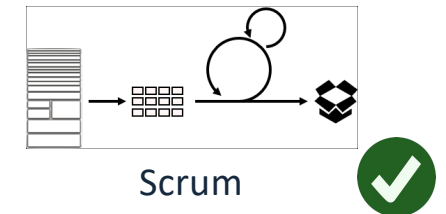
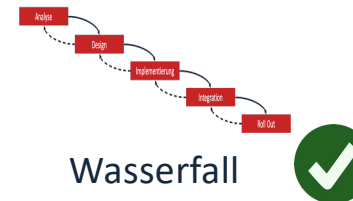
UML ist eine **Notation**, um Softwaresysteme zu **analysieren** und zu **designen**.



Analyse

Design

UML ist **kein** Vorgehensmodell und erzwingt keine bestimmte Vorgehensweise.

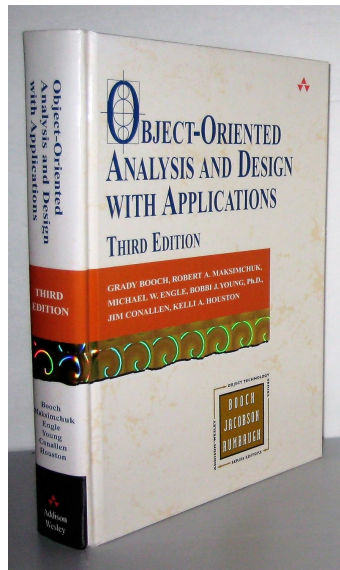


UML unterstützt insbesondere **objektorientierte Ansätze**.

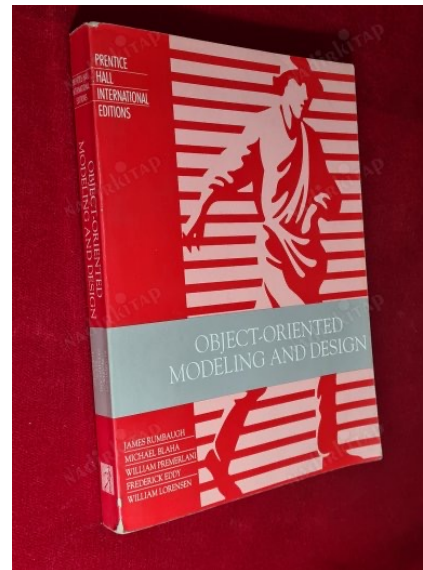


Unified Modeling Language (UML)

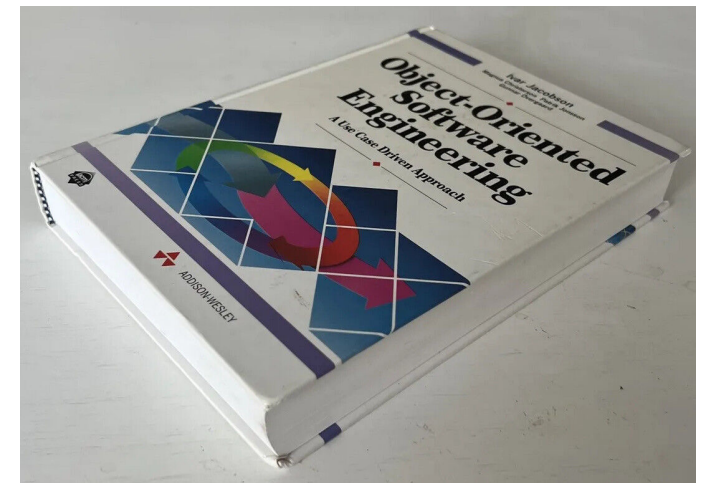
Was ist daran "unified"?



Grady Booch
(1990)



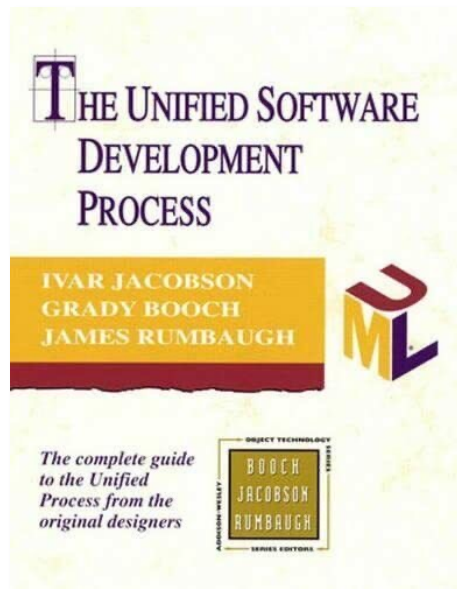
James Rumbaugh
(1991)



Ivar Jacobson
(1992)

Unified Modeling Language (UML)

Was ist daran "unified"?



Grady Booch,
James Rumbaugh,
Ivar Jacobson
(1999)

Die drei "Amigos" waren seit 1994 im Unternehmen **Rational Software**.

- Entwicklung und Consulting rund um das Vorgehensmodell **Unified Process** (heute kaum mehr benutzt)
- Dabei Zusammenführen der bis dahin unterschiedlichen Notationen zur **Unified Modeling Language**

Unified Modeling Language (UML)

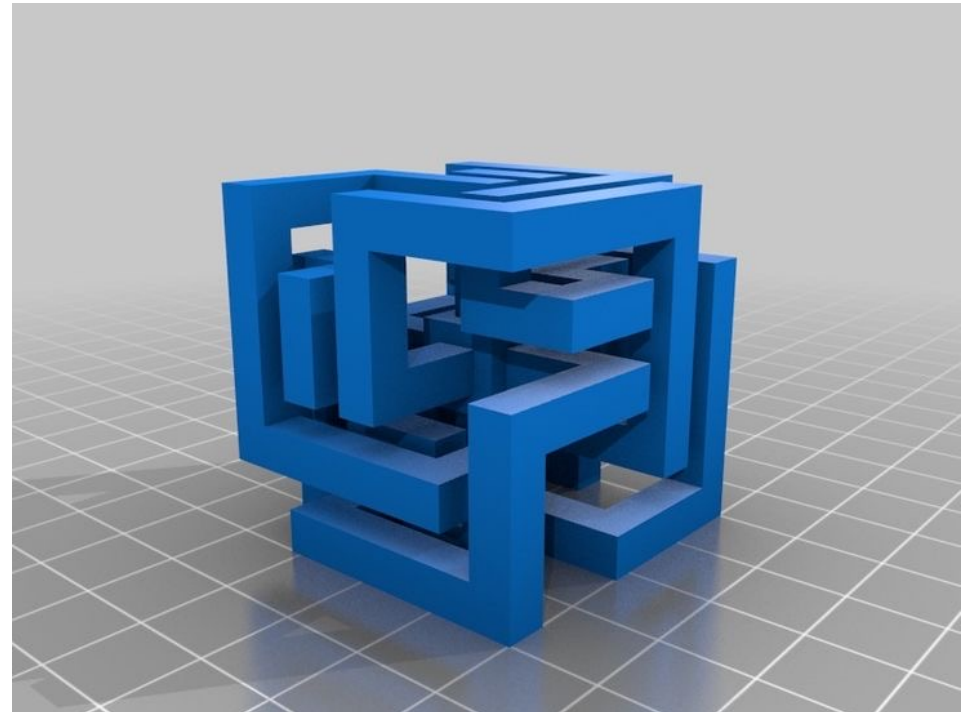
Unterschiedliche Sichten auf Software

Softwaresysteme sind oft so komplex, dass das Betrachten nur aus einem Blickwinkel nicht ausreicht, um das Ganze zu verstehen.

Die **UML** bietet daher **13 Diagrammtypen**, die jeweils einen anderen Aspekt des Systems betrachten.

Diagrammtypen, die den **statischen Aufbau** der Software modellieren, heißen **Strukturdiagramme**.

Diagrammtypen, die das **dynamische Verhalten** der Software beschreiben, heißen **Verhaltensdiagramme**.



Unified Modeling Language (UML)

Unterschiedliche Sichten auf Software

Strukturdiagramme

- Objektdiagramme
- Klassendiagramme
- Komponentendiagramme
- Verteilungsdiagramme
- Paketdiagramme
- Kompositionsstrukturdiagramme

Verhaltensdiagramme

- Use Case Diagramme
- Aktivitätsdiagramme
- Zustandsdiagramme
- Zeitdiagramme
- Kommunikationsdiagramme
- Sequenzdiagramme
- Interaktionsübersichtsdiagramme

Unified Modeling Language (UML)

Unterschiedliche Sichten auf Software

Strukturdiagramme

- Objektdiagramme
- **Klassendiagramme**
- Komponentendiagramme
- Verteilungsdiagramme
- Paketdiagramme
- Kompositionsstrukturdiagramme

Verhaltensdiagramme

- Use Case Diagramme
- Aktivitätsdiagramme
- Zustandsdiagramme
- Zeitdiagramme
- Kommunikationsdiagramme
- **Sequenzdiagramme**
- Interaktionsübersichtsdiagramme

Unified Modeling Language (UML)

Modellelemente

Ein UML-Diagramm besteht i.d.R. aus **Modellelementen** als graphischen "Bausteinen" und Verbindungen (z.B. Pfeilen) zwischen den Modellelementen.

Je nach Diagrammtyp können **die Verbindungen und Pfeile** unterschiedliche Semantik (=Bedeutung) besitzen.

Nicht jedes **Modellelement** ist in jedem Diagramm zulässig. Falls es aber zulässig ist, besitzt es in allen UML-Diagrammen die gleiche graphische Darstellung.

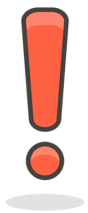


Klassendiagramme

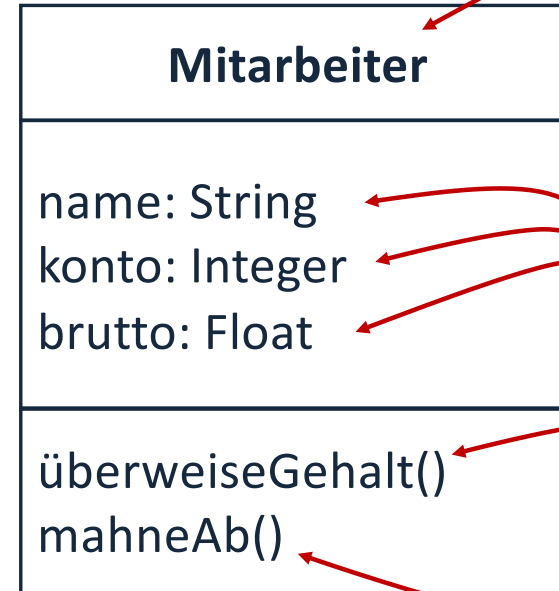
Klassendiagramme

Modellelement "Klasse"

Das wichtigste Modellelement in einem Klassendiagramm ist die **Klasse**.



Die Angabe von Attributen und Methoden ist optional und kann z.B. in frühen Stadien der Modellierung weggelassen werden.



Klassenname
fett,
mit Großbuchstaben
beginnend

Attribute
mit Datentyp (optional)

Methoden

Klassendiagramme

Modellelement "Klasse" - Sichtbarkeit

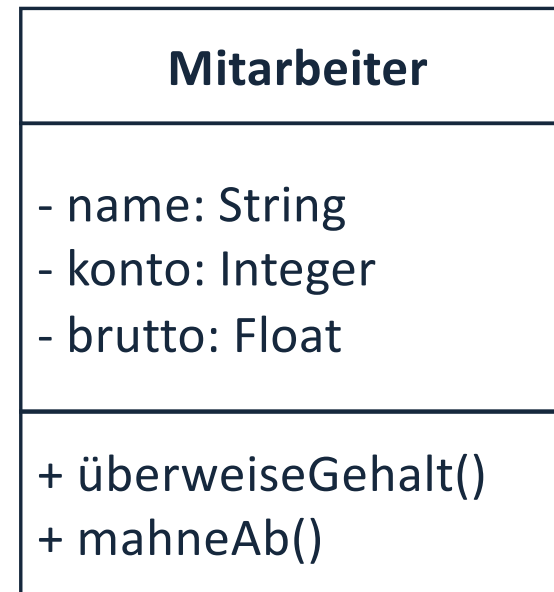
Durch ein Präfix vor dem Attribut- bzw. Methodennamen kann die Sichtbarkeit festgelegt werden:

- name : String

Das Attribut ist nur innerhalb der Klasse sichtbar (privat)

+ überweiseGehalt()

Die Methode ist auch für Instanzen anderer Klassen sichtbar (öffentlich)



Klassendiagramme

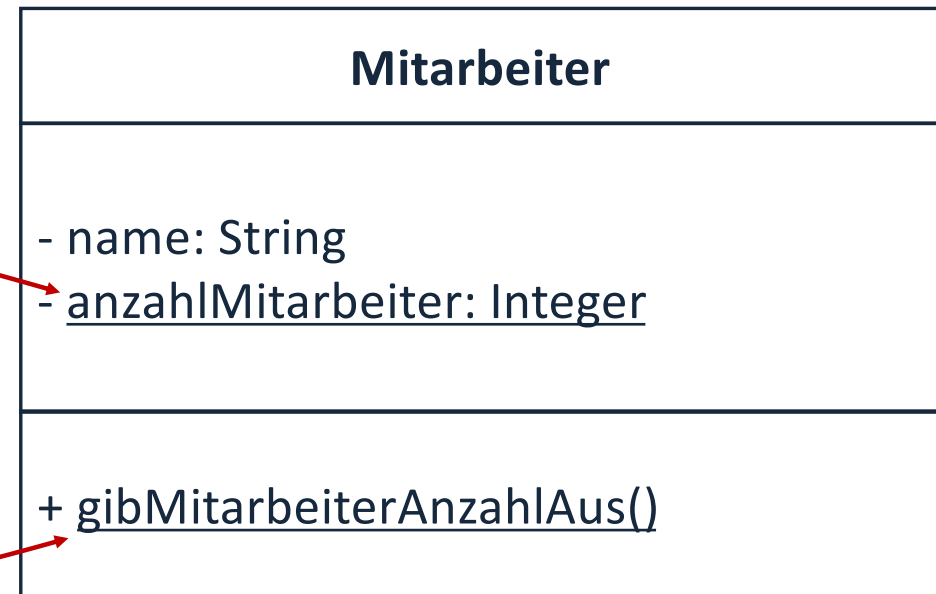
Modellelement "Klasse" – Klassenattribute und Klassenmethoden

Klassenattribute und Klassenmethoden werden in UML unterstrichen.

Klassenattribut

ist der Klasse zugeordnet,
nicht dem einzelnen Mitarbeiter

Klassenmethode



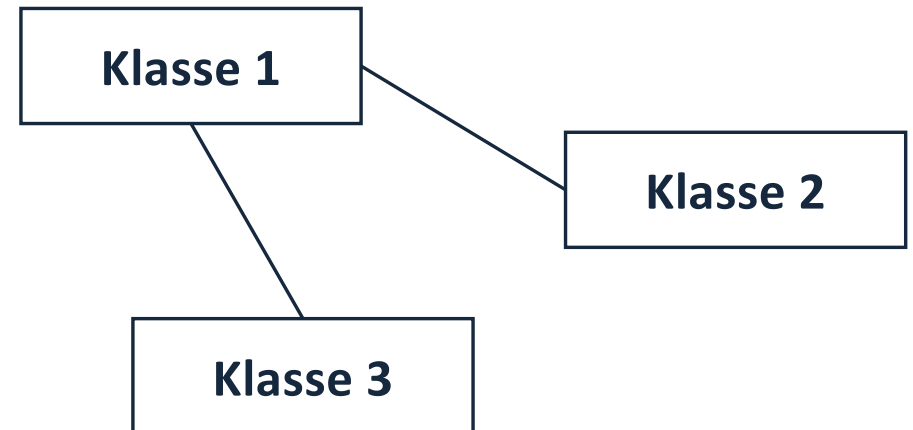
In Java: static!

Klassendiagramme

Beziehungen zwischen Klassen

Im Klassendiagramm werden die **Beziehungen** zwischen Klassen durch Linien zwischen den Klassen dargestellt.

Klassendiagramm



Klassendiagramme

Beziehungen zwischen Klassen – Assoziation

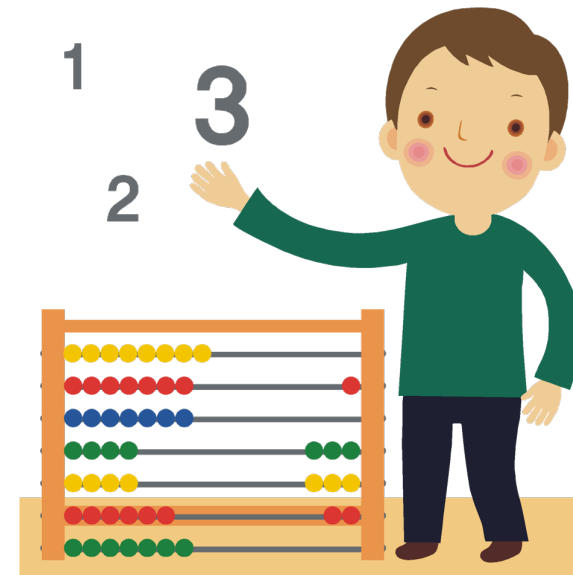
Mit einer einfachen Linie wird festgehalten, dass es einen Zusammenhang zwischen diesen Klassen gibt. Eine solche Verbindung heißt **Assoziation**.



Klassendiagramme

Beziehungen zwischen Klassen – Kardinalitäten

1	genau eines
0..2	null bis zwei
*	größer oder gleich null
2..*	größer oder gleich zwei



Klassendiagramme

Beziehungen zwischen Klassen – Aggregation

Sonderfall einer Assoziation: Sind die Instanzen der einen Klasse Bestandteile der Instanz der anderen Klasse, so handelt es sich um eine **Aggregation**. Eine Name ist für die Verbindung nicht notwendig, weil sie implizit als **besitzt (has-a)** gelesen werden kann.

Auto besitzt 4 Reifen:



Aggregation wird durch eine Raute dargestellt

Klassendiagramme

Beziehungen zwischen Klassen – Komposition

Die **Komposition** geht einen Schritt weiter als die Aggregation: Hier können die Komponenten ohne das Aggregat nicht existieren. Eine Name ist für die Verbindung wieder nicht notwendig, weil sie weiterhin implizit als **besitzt (has-a)** gelesen werden kann.



Bei einer Komposition ist die Raute ausgefüllt

Klassendiagramme

Beziehungen zwischen Klassen – Vererbung / Generalisierung

Ist eine Klasse ein Spezialfall einer anderen Klasse, so kann dies durch eine **Vererbungsbeziehung** visualisiert werden. Die Verbindung kann in diesem Fall implizit mit **ist-ein (is-a)** gelesen werden. Damit ist es zwangsweise eine 1-1 Beziehung (keine Kardinalitäten notwendig).



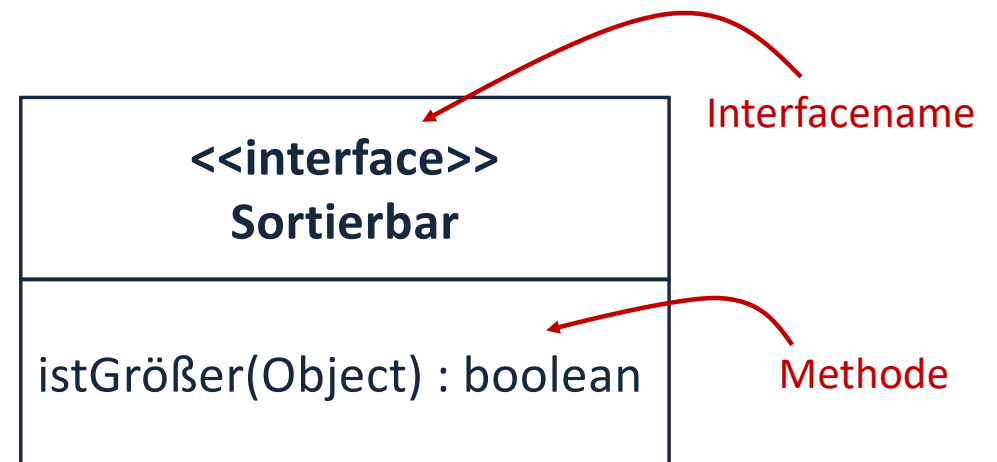
Eine Vererbungsbeziehung erkennt man am nicht ausgefüllten Dreieck bei der Oberklasse

Klassendiagramme

Modellelement "Interface"

Ein **Interface** wird wie eine Klasse, aber mit dem Schlüsselwort <<interface>> dargestellt.

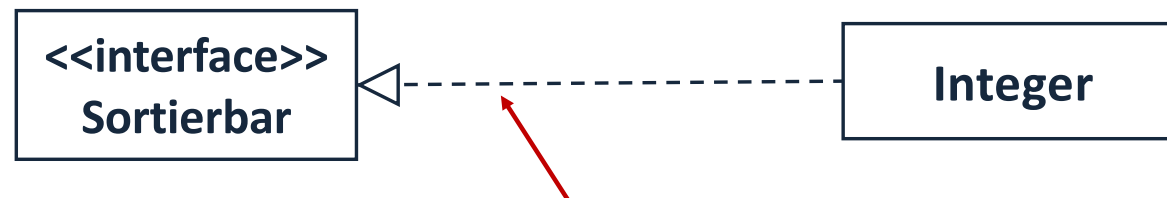
Interfaces besitzen keine Attribute, sondern lediglich Methodensignaturen.



Klassendiagramme

Implementierung eines Interfaces

Implementiert eine Klasse ein **Interface**, so kann dies ebenfalls im Klassendiagramm dargestellt werden.



Die gestrichelte Line drückt aus, dass das Interface implementiert wird.

Klassendiagramme

Beispiel

Ein Unternehmen besteht aus mehreren Abteilungen. Jeder Mitarbeiter des Unternehmens ist genau einer Abteilung zugeordnet. Das Unternehmen führt mehrere Projekte durch. Jedes Projekt hat genau einen Mitarbeiter als Projektleiter. Alle Mitarbeiter des Unternehmens können in Projekten mitarbeiten und auch als Projektleiter eingesetzt werden – beides auch parallel in mehreren Projekten. Für ein Projekt ist immer genau eine Abteilung verantwortlich, die auch mehrere Projekte verantworten kann. Jedes Projekt betrifft mindestens eine Abteilung.

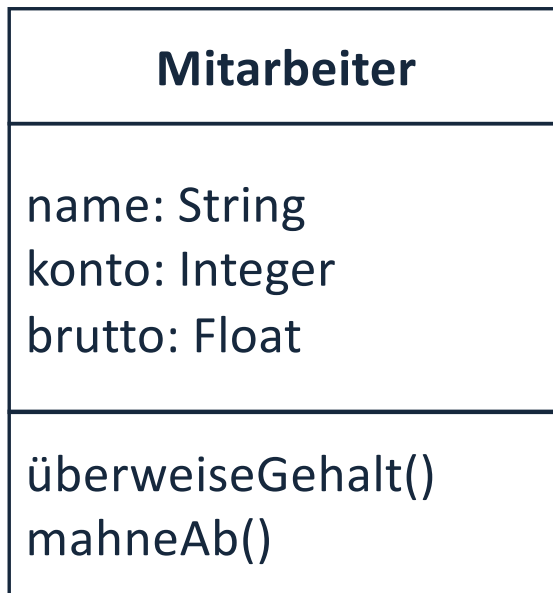
Erstellen Sie hierzu ein Klassendiagramm.

Sequenzdiagramme

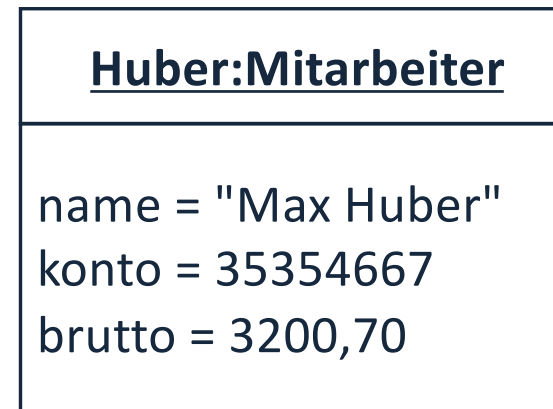
Sequenzdiagramme

Modellelement "Objekt"

Klasse

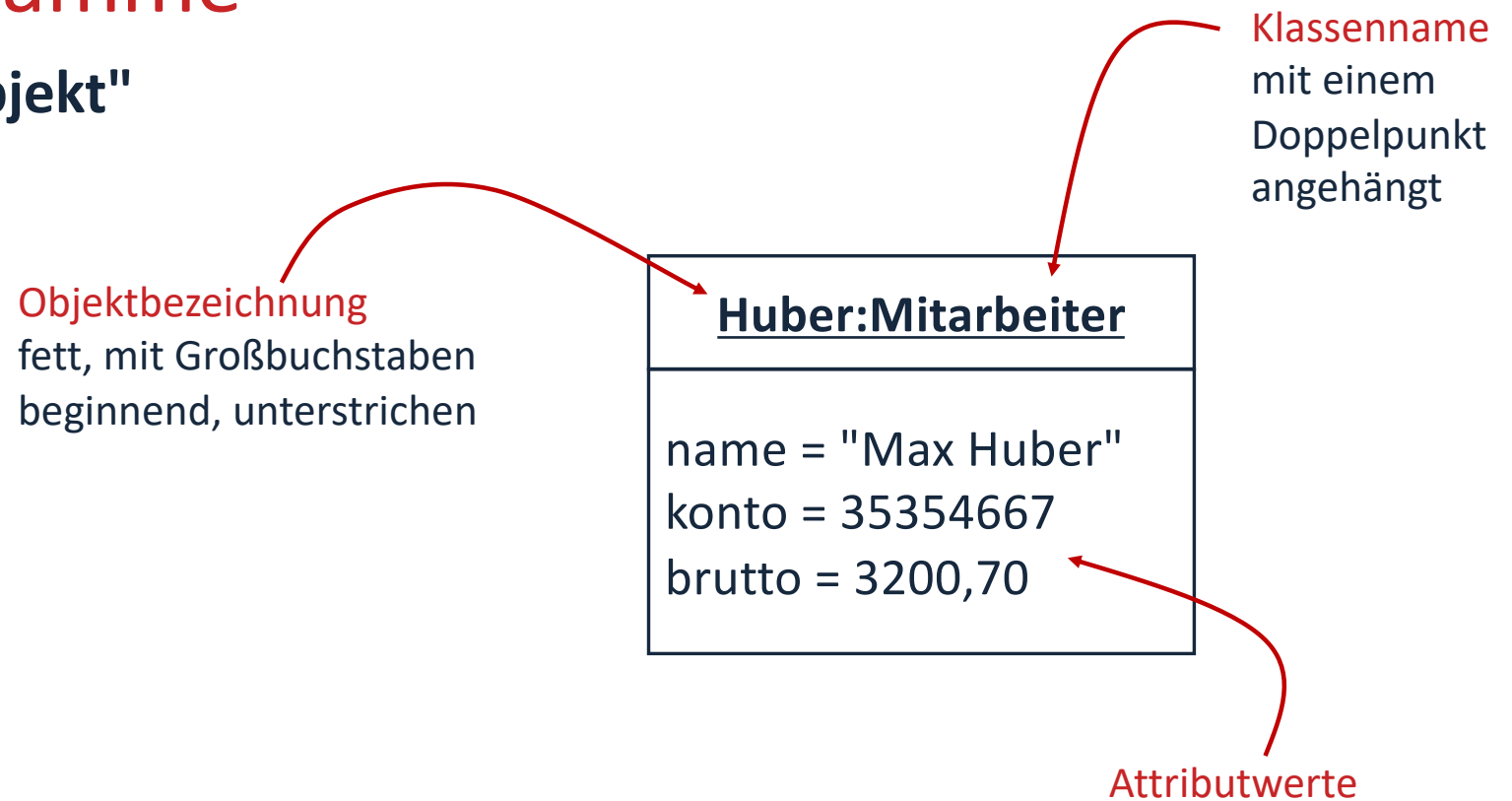


Objekt



Sequenzdiagramme

Modellelement "Objekt"



Sequenzdiagramme

Modellelement "Objekt"

Huber:Mitarbeiter

Zulässig:

Attributwerte müssen nicht angegeben werden

:Mitarbeiter

Zulässig:

Objektbezeichnung kann entfallen; gemeint ist beliebige Instanz

Huber

Zulässig:

Klassenname kann entfallen; Klasse ergibt sich aus dem Kontext

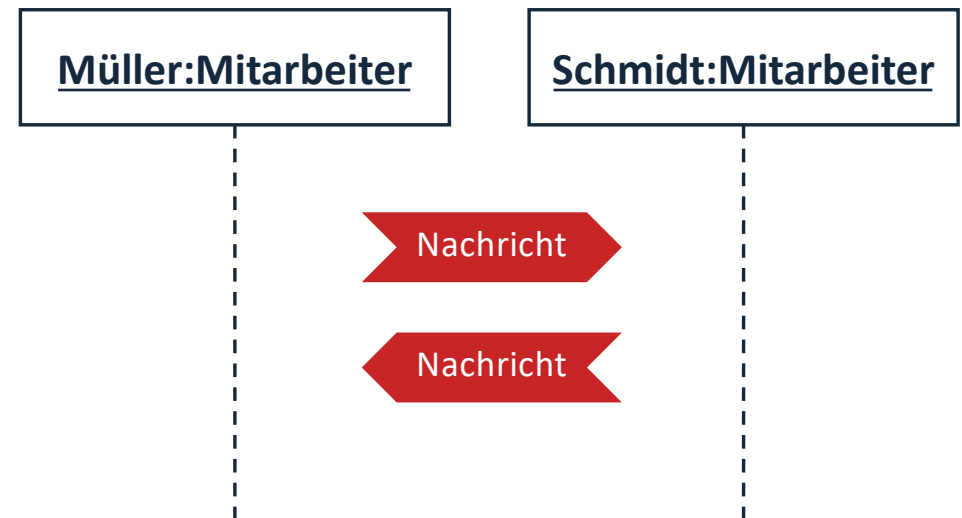
Aber: Objektbezeichnung *oder* Klassenname müssen angegeben werden!

Sequenzdiagramme

Nachrichten zwischen Objekten

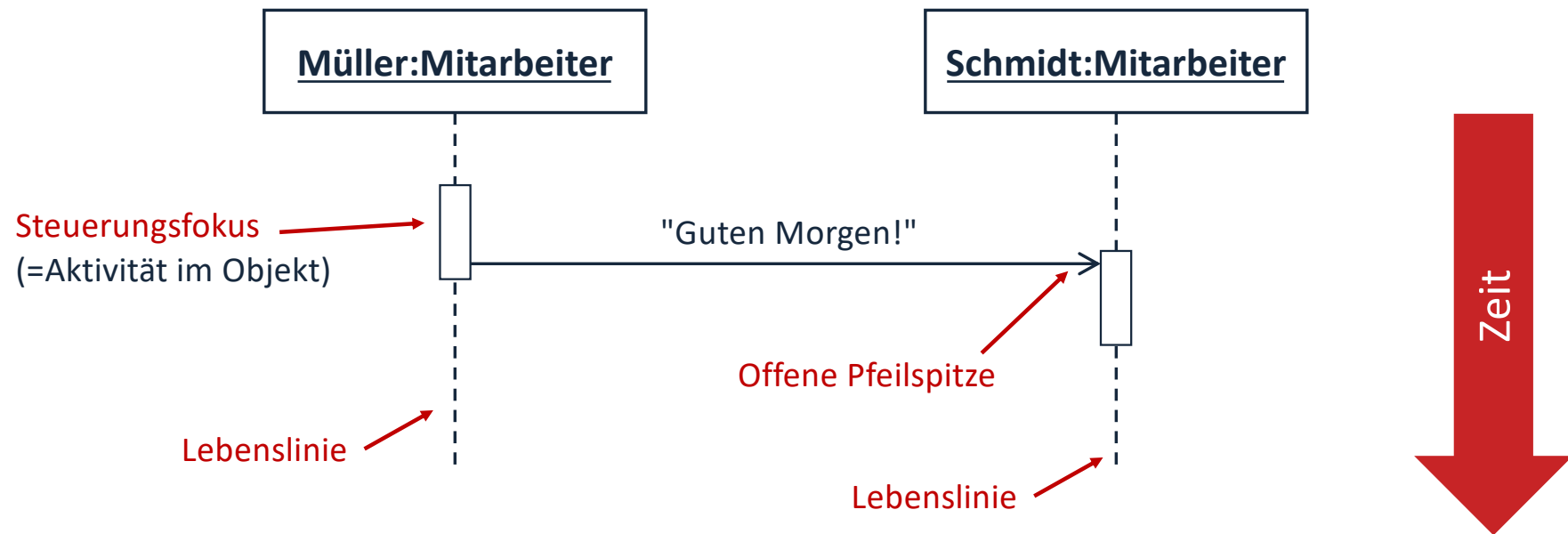
In einem Sequenzdiagramm beschreiben wir einen ***möglichen*** Nachrichtenaustausch zwischen ***Objekten*** in einem ***Zeitfenster*** zur Laufzeit unseres Systems.

Sequenzdiagramm



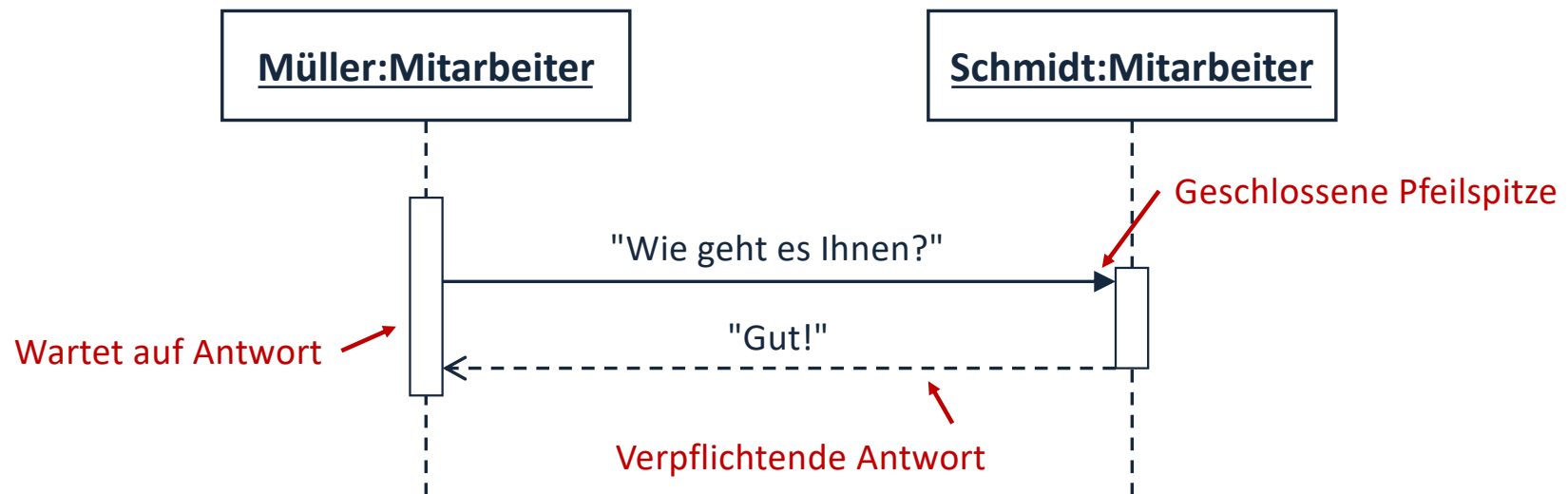
Sequenzdiagramme

Asynchrone Nachricht



Sequenzdiagramme

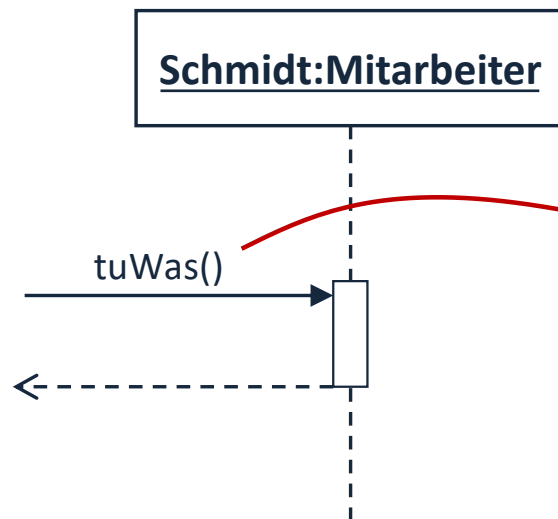
Synchrone Nachricht



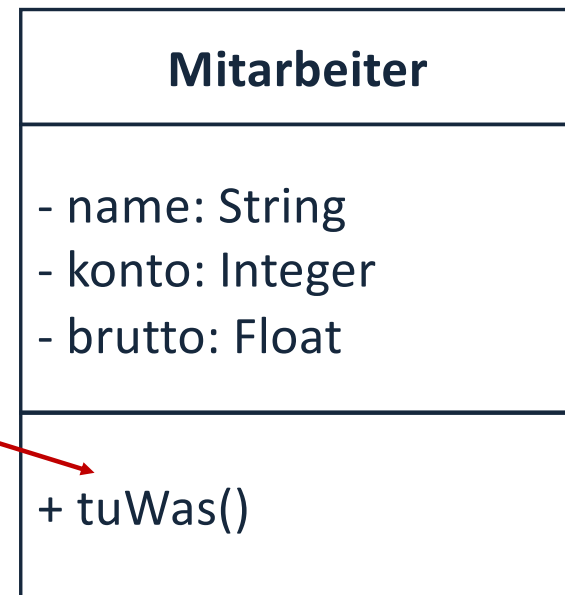
Sequenzdiagramme

Nachrichten triggern Methoden

Sequenzdiagramm

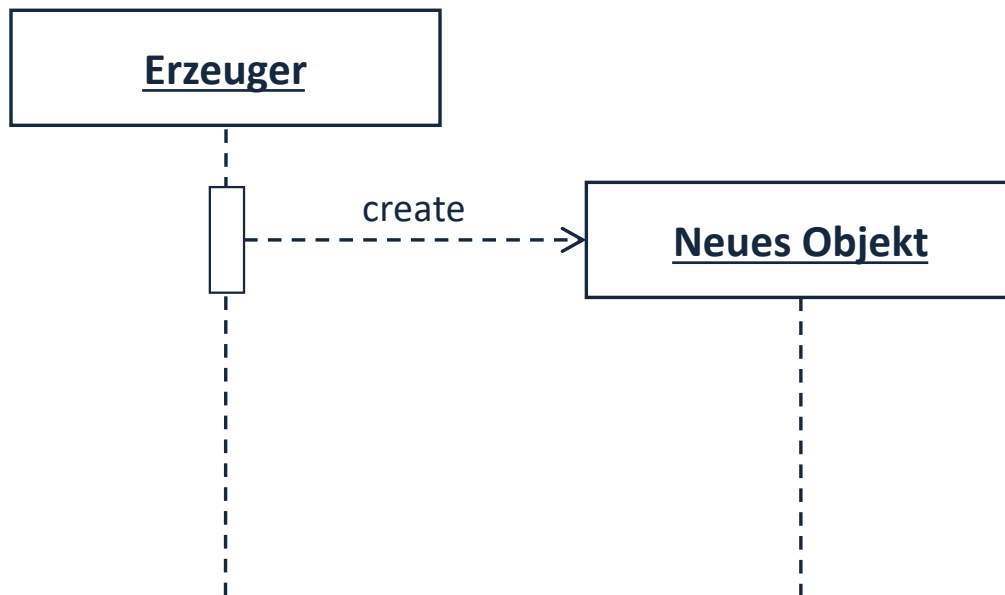


Klassendiagramm



Sequenzdiagramme

Objekt-Konstruktion



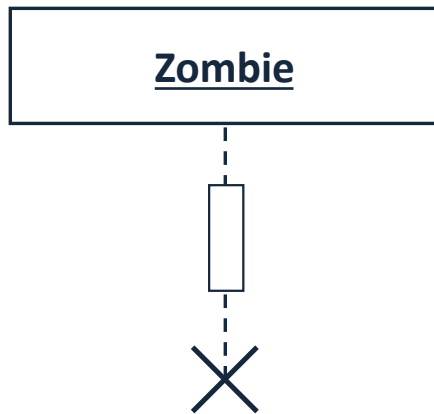
Existiert ein Objekt am Beginn des betrachteten Zeitfensters noch nicht, kann es dennoch während des Ablaufs ***konstruiert*** werden.

Es wird dann "weiter unten" (d.h. zeitlich später) eingezeichnet.

Die Erzeugung wird mit einem gestrichelten Pfeil mit offener Pfeilspitze modelliert.

Sequenzdiagramme

Objekt-Destruktion



Endet die Lebensdauer eines Objekts während des betrachteten Zeitfensters, dann endet auch die **Lebenslinie** in einem Kreuz.

Sequenzdiagramme

Beispiel

In einem Flughafen müssen Starts und Landungen für Flüge gemanagt werden. Dazu sendet der Tower an einen Piloten die Benachrichtigung zum bevorstehenden Start. Der Pilot antwortet an den Tower. Daraufhin erhält der Pilot vom Tower die Startfreigabe. Er checkt daraufhin asynchron das Flugzeug-Kontrollsystem. Außerdem gibt er an seine Fluggesellschaft die Info weiter, dass der Flug freigegeben ist. Meldet das Flugzeug-Kontrollsystem, dass es bereit ist, bestätigt der Pilot als Antwort auf die Startfreigabe an den Tower, dass er fertig zum Abflug ist.

Stellen Sie die Kommunikation mit einem Sequenzdiagramm dar (Aufrufe sind synchron falls nicht anders erwähnt). Verwenden sie sinnvolle Operationsnamen für die Aufrufe.

Vielen Dank für die Aufmerksamkeit!

Fragen?